EXPERIMENT - 7

AIM

To identify functional dependencies in a denormalized Sales table and normalize it step by step from First Normal Form (1NF) to Third Normal Form (3NF) and Boyce–Codd Normal Form (BCNF). Verify lossless join and dependency preservation using SQL queries.

1. Create the Database and Denormalized Table

Explanation: A denormalized table stores customer, salesperson, product, and sales details together. This may cause data redundancy because customer and salesperson information is repeated across multiple rows.

```
CREATE DATABASE IF NOT EXISTS SalesDB;
USE SalesDB;
DROP TABLE IF EXISTS Sales;
CREATE TABLE Sales (
sale_id INT, sale_date DATE, customer_id INT, customer_name VARCHAR(50),
salesperson_id INT, salesperson_name VARCHAR(50), product_id INT,
product_name VARCHAR(50), quantity INT
);
-- Insert sample records
INSERT INTO Sales VALUES
(1001, '2026-01-10', 1, 'Aarav', 101, 'Rahul', 201, 'Laptop', 1),
(1001, '2026-01-10', 1, 'Aarav', 101, 'Rahul', 202, 'Mouse', 2),
(1002, '2026-01-11', 2, 'Diya', 102, 'Priya', 201, 'Laptop', 1),
(1002, '2026-01-11', 2, 'Diya', 102, 'Priya', 203, 'Keyboard', 1),
(1003, '2026-01-12', 1, 'Aarav', 101, 'Rahul', 202, 'Mouse', 3),
(1003, '2026-01-12', 1, 'Aarav', 101, 'Rahul', 204, 'Monitor', 1);
-- Display denormalized data
SELECT * FROM Sales;
```

2. Identify Functional Dependencies

Explanation: A functional dependency describes how one attribute determines another attribute. The functional dependencies for this dataset are:

Functional Dependency	**Meaning**
sale_id → sale_date, customer_id, salesperson_id	Each sale determines its date, customer, and salesperson.
customer_id → customer_name	Each customer ID determines the customer's name.

salesperson_id → salesperson_name	Each salesperson ID determines the salesperson name.
product_id → product_name	Each product ID determines its name.
(sale_id, product_id) → quantity	A sale and product combination determines the quantity sold.

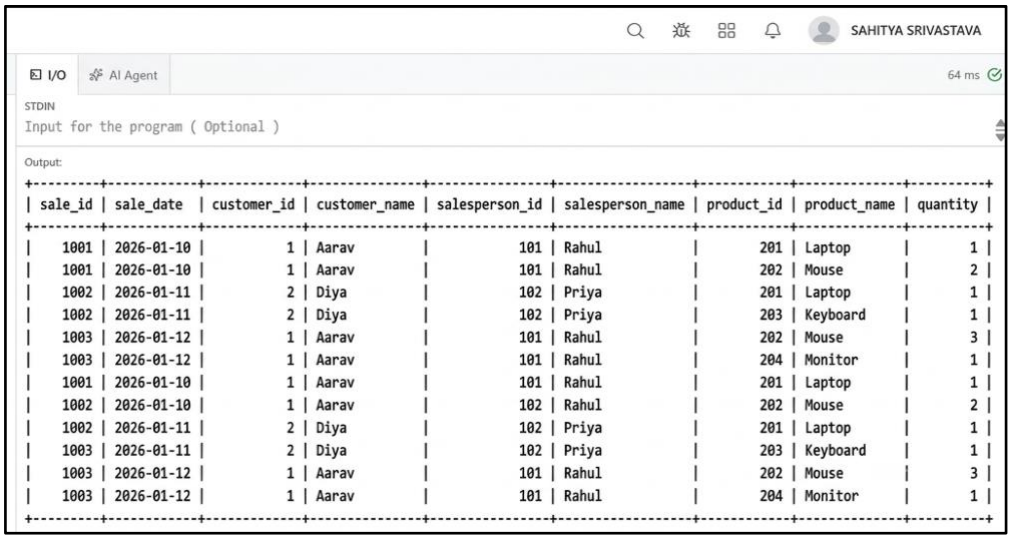
Candidate key of the 1NF table: (sale_id, product_id).

3. Convert the Table into First Normal Form (1NF)

Explanation: A table is in 1NF when each field contains a single value and there are no repeating groups. Each row in the provided Sales table contains one product for one sale, so it satisfies 1NF.

```
-- Display 1NF records
SELECT sale_id, sale_date, customer_id,
customer_name,       salesperson_id,
salesperson_name,       product_id, product_name,
quantity
FROM Sales
ORDER BY sale_id, product_id;
```

Output: Six rows, with each row representing one product within a sale.



sale_id	sale_date	customer_id	customer_name	salesperson_id	salesperson_name	product_id	product_name	quantity
1001	2026-01-10	1	Aarav	101	Rahul	201	Laptop	1
1001	2026-01-10	1	Aarav	101	Rahul	202	Mouse	2
1002	2026-01-11	2	Diya	102	Priya	201	Laptop	1
1002	2026-01-11	2	Diya	102	Priya	203	Keyboard	1
1003	2026-01-12	1	Aarav	101	Rahul	202	Mouse	3
1003	2026-01-12	1	Aarav	101	Rahul	204	Monitor	1
1001	2026-01-10	1	Aarav	101	Rahul	201	Laptop	1
1002	2026-01-10	1	Aarav	102	Rahul	202	Mouse	2
1002	2026-01-11	2	Diya	102	Priya	201	Laptop	1
1003	2026-01-11	2	Diya	102	Priya	203	Keyboard	1
1003	2026-01-12	1	Aarav	101	Rahul	202	Mouse	3
1003	2026-01-12	1	Aarav	101	Rahul	204	Monitor	1

4. Convert the Table into Second Normal Form (2NF)

Explanation: A table is in 2NF when it is in 1NF and every non-key attribute depends on the whole candidate key, not just part of a composite key.
The key (sale_id, product_id) has partial dependencies:

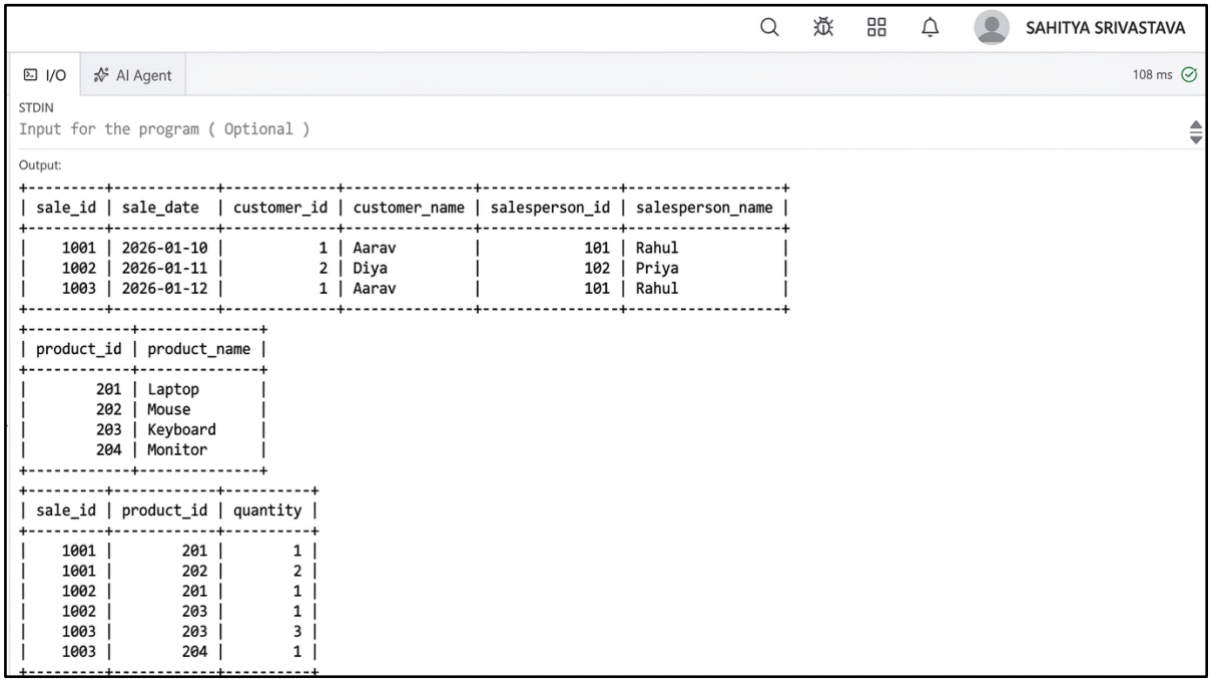
- sale_id determines sale-level details.
- product_id determines product details.

To remove these partial dependencies, separate the data into Sale_Header, Product, and Sale_Item.

```
-- Create sale header table
CREATE TABLE Sale_Header AS
SELECT DISTINCT sale_id, sale_date, customer_id, customer_name,
salesperson_id, salesperson_name
```

```
FROM Sales;
-- Create product table
CREATE TABLE Product AS
SELECT DISTINCT product_id, product_name
FROM Sales;
-- Create sale item table
CREATE TABLE Sale_Item AS
SELECT sale_id, product_id, quantity
FROM Sales;
-- Display normalized tables
SELECT * FROM Sale_Header ORDER BY sale_id;
SELECT * FROM Product ORDER BY product_id;
SELECT * FROM Sale_Item ORDER BY sale_id, product_id;
```

Output: Sale_Header: 3 rows, Product: 4 rows Sale_Item: 6 rows



5. Convert the Tables into Third Normal Form (3NF)

Explanation: A table is in 3NF when it is in 2NF and has no transitive dependency of non-key attributes on the primary key. In Sale_Header, the customer name depends on customer_id, and the salesperson name depends on salesperson_id. These details should be stored in separate tables.

```
-- Create customer table
CREATE TABLE Customer AS
SELECT DISTINCT customer_id, customer_name
```

```
FROM Sale_Header;
-- Create salesperson table
CREATE TABLE Salesperson AS
SELECT DISTINCT salesperson_id, salesperson_name
FROM Sale_Header;
-- Create normalized sale header
CREATE TABLE Sale_Header_3NF AS
SELECT sale_id, sale_date, customer_id, salesperson_id
FROM Sale_Header;
-- Display 3NF tables
SELECT * FROM Customer ORDER BY customer_id;
SELECT * FROM Salesperson ORDER BY salesperson_id;
SELECT * FROM Sale_Header_3NF ORDER BY sale_id;
```

Output: Customer: 2 rows, Salesperson: 2 rows, Salesperson: 2 rows.



6. Verify the Boyce–Codd Normal Form (BCNF)

Explanation: BCNF requires every determinant in a non-trivial functional dependency to be a super key. Under the functional dependencies identified in this experiment, the decomposed tables satisfy BCNF because each determinant is a candidate key in its respective table.

```
-- Verify customer records
SELECT customer_id, customer_name
FROM Customer;
```

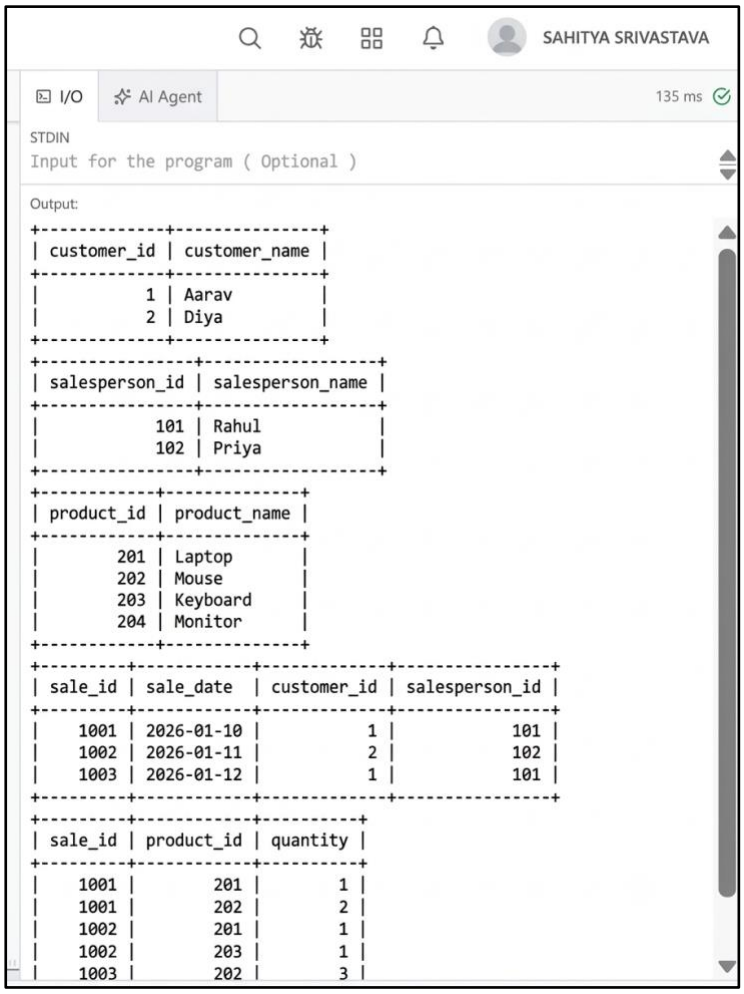
```
-- Verify salesperson records
SELECT salesperson_id, salesperson_name
FROM Salesperson;

-- Verify product records
SELECT product_id, product_name
FROM Product;

-- Verify sale header
SELECT sale_id, sale_date, customer_id, salesperson_id
FROM Sale_Header_3NF;

-- Verify sale items
SELECT sale_id, product_id, quantity
FROM Sale_Item;
```

Output: The normalized tables display the separated customer, salesperson, product, sale-header, and sale-item information.



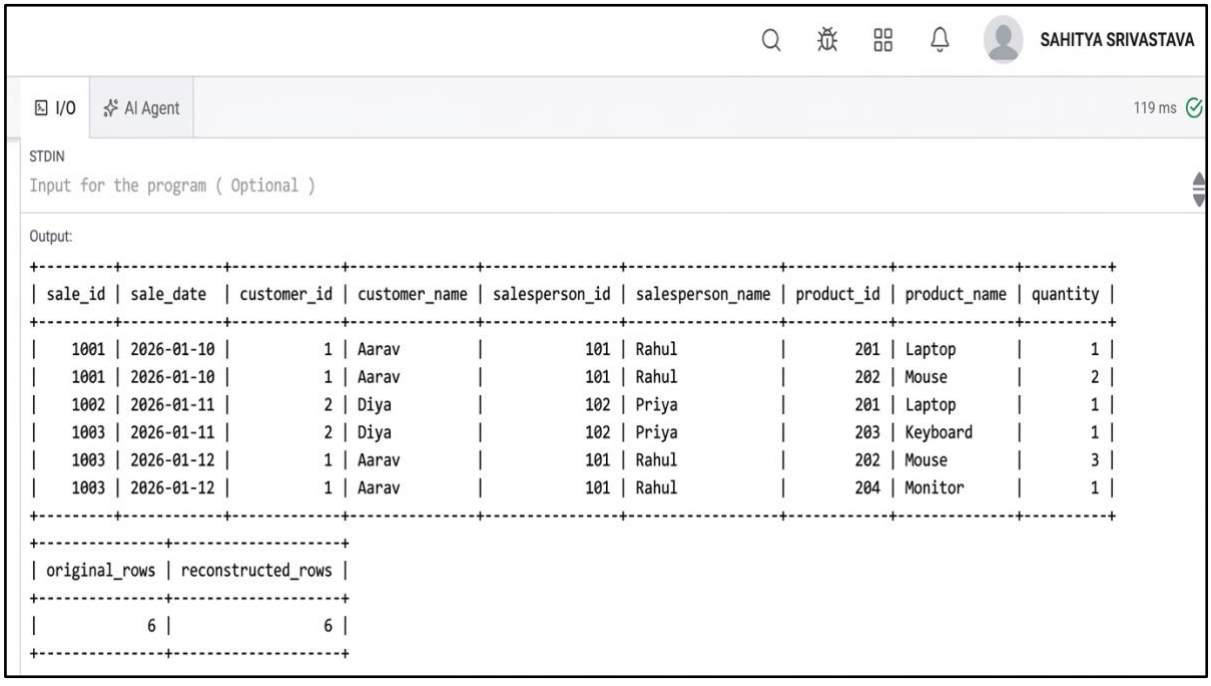
7. Verify Lossless Join

Explanation: A decomposition is lossless when joining the decomposed tables reconstructs the original records without losing or creating rows.

```
-- Reconstruct original sales data
SELECT H.sale_id, H.sale_date, H.customer_id, C.customer_name,
      H.salesperson_id, SP.salesperson_name, P.product_id, P.product_name, I.quantity
FROM Sale_Header_3NF H
JOIN Customer C ON H.customer_id = C.customer_id
JOIN Salesperson SP ON H.salesperson_id = SP.salesperson_id
JOIN Sale_Item I ON H.sale_id = I.sale_id
JOIN Product P ON I.product_id = P.product_id
ORDER BY H.sale_id, P.product_id;

-- Compare row counts
SELECT (SELECT COUNT(*) FROM Sales) AS original_rows,
      (SELECT COUNT(*) FROM Sale_Header_3NF H
       JOIN Customer C ON H.customer_id = C.customer_id
       JOIN Salesperson SP ON H.salesperson_id = SP.salesperson_id
       JOIN Sale_Item I ON H.sale_id = I.sale_id
       JOIN Product P ON I.product_id = P.product_id) AS reconstructed_rows;
```

Output: The reconstructed result contains the same six sales-line records. The count comparison should show original_rows = 6 and reconstructed_rows = 6.



8. Verify Dependency Preservation

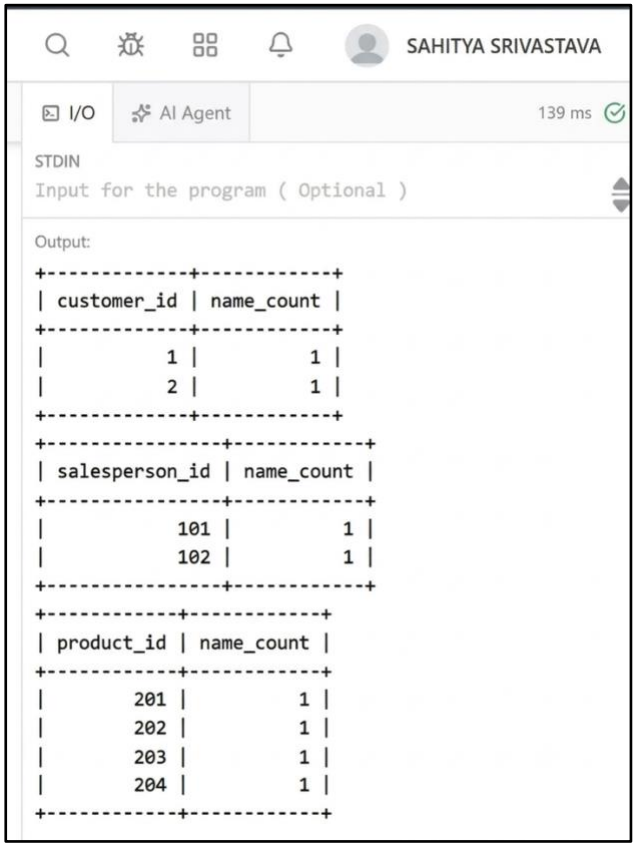
Explanation: A decomposition preserves dependencies when the original functional dependencies can be checked using the decomposed tables without requiring the tables to be joined. The following queries verify that the customer, salesperson, and product identifiers uniquely identify their corresponding names.

```
-- Check customer dependency
SELECT customer_id, COUNT(DISTINCT customer_name) AS name_count
FROM Customer
GROUP BY customer_id;

-- Check salesperson dependency
SELECT salesperson_id, COUNT(DISTINCT salesperson_name) AS name_count
FROM Salesperson
GROUP BY salesperson_id;

-- Check product dependency
SELECT product_id, COUNT(DISTINCT product_name) AS name_count
FROM Product
GROUP BY product_id;
```

Output: Each identifier should have name_count = 1, demonstrating that each ID maps to one name in the sample data.



RESULT

The denormalized Sales table was analyzed using functional dependencies and decomposed into normalized tables. The decomposition was checked using SQL queries for reconstructed row counts and dependency consistency.

CONCLUSION

Normalization reduces data redundancy and helps prevent insertion, update, and deletion anomalies. The experiment demonstrates the principles of 1NF, 2NF, 3NF, and BCNF, along with lossless reconstruction and dependency checks.